# System Test Plan

**Testing Overview: Template**
**Test Data Summary:**

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC x] 3.x.x.i Test** | Preconditions: | | |

**Testing Overview:** The customer selects the desired storefront, can select one or more recipes/items to add to their order, and add a tip.
**Test Data Summary:** Customer is logged in. Meat ingredient is in inventory. Burger item is on the menu at $10.

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 1] 3.1.1.1 Test Valid Customer Order** | Preconditions: User is logged in as a customer. A recipe named “Burger” has been added to a location called “Raleigh” by a staff member allocated to the location.<br>1. Navigate to “Order” page<br>2. The menu is displayed with “Add to Order” buttons<br>3. Select “Raleigh” for location<br>4. Add “Burger” to the cart<br>5. Select 20% tip<br>6. Tax Rate is applied<br>7. Click “Create Order” and check results | “Order created successfully!” message is shown to the user. User can see their order by going to order pickup for anonymous users tab and entering their order id and email, and staff associated with “Raleigh” can see the order in their orders tab | “Order created successfully!” message is shown to the user. User can see their order by going to order pickup tab, and staff associated with “Raleigh” can see the order in their orders tab with the “Burger” item, pending status, etc. |

**Testing Overview:** The customer selects the desired storefront, can select one or more recipes/items to add to their order, and add a tip.
**Test Data Summary:** Customer is logged in. Meat ingredient is in inventory. Burger item is on the menu at $10.

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 1] 3.1.1.2 Test Insufficient Ingredients Customer Order** | Preconditions: User is logged in as a customer. A recipe named “Burger” has been added to a location called “Raleigh” by a staff member allocated to the location. There are not enough of the ingredients in the inventory to make Burger.<br>1. Navigate to “Order” page<br>2. The menu is displayed with “Add to Order” buttons<br>3. Select “Raleigh” for location<br>4. Add “Burger” to the cart<br>5. Select 20% tip<br>6. Tax Rate is applied<br>7. Click “Create Order” and check results | The user is able to place an order in the service. However, when a staff tries to fulfill the order they will not be able to mark it as ready for pickup. | The user is able to create the “Burger” order; however, the staff user associated with Raleigh will be unable to fulfill the order as ready for pickup. |

**Testing Overview:** The customer selects the desired storefront, can select one or more recipes/items to add to their order, and add a tip.

**Test Data Summary:** Customer is logged in. Meat ingredient is in inventory. Burger item is on the menu at $10.

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 1] 3.1.1.3 Test Anonymous Customer Order** | Preconditions: User is not logged in and wants to place an anonymous order. A recipe named “Burger” has been added to a location called “Raleigh” by a staff member allocated to the location.<br>1. Navigate to “Order” page<br>2. The menu is displayed with “Add to Order” buttons<br>3. Select “Raleigh” for location<br>4. Enter a valid email address for | Transaction is canceled and a message saying “Lack of funds” is displayed. | “Order created successfully!” message is shown to the user. User can see their order by going to order pickup for anonymous users tab and entering their order id and email, and staff associated with “Raleigh” can see the order in their orders tab. User can see the contents of their order, price of their order, status of their order, etc. |

| | | | |
|---|---|---|---|
| | order email<br>5. Add “Burger” to the cart<br>6. Select 20% tip<br>7. Tax Rate is applied<br>8. Click “Create Order” and check results | | |

**Testing Overview:** Launch the backend, launch the frontend, open the api webpage.
**Test Data Summary:** Meat ingredient is in inventory, Lettuce ingredient is in inventory, Bun ingredient is in inventory

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 2] 3.2.1.1 Test Add to Menu** | Preconditions: User is logged in as a staff member<br>1. Navigate to “Menu” page<br>2. Click “Add Item to Menu”<br>3. Type “Water Bottle” for name, input a valid description, 5.00 for price, and 100 for amound=t<br>4. Submit the form, and check results | The “Water bottle” appears in the menu, with 5 price and a quantity of 100 in inventory. | The “Water bottle” appears in the menu, with 5 price and a quantity of 100 in inventory. |

**Testing Overview:** Launch the backend, launch the frontend, open the api webpage.
**Test Data Summary:** Meat ingredient is in inventory, Lettuce ingredient is in inventory, Bun ingredient is in inventory

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 2] 3.2.1.2 Test Add Invalid Item Amount** | Preconditions: User is logged in as a staff member<br>1. Navigate to “Menu” page<br>2. Click “Add Item to Menu”<br>3. Type “InvalidBurger” for name, 500 for price, and -1 Meat, 1 Lettuce, 1 Bun<br>4. Submit the form, and check results | The form should not submit, as there was an invalid unit for ingredient amounts | The system delivers the prompt, “Amount must be valid integer.” |

| | | | |
|---|---|---|---|
| | | | |

**Testing Overview:** Launch the backend, launch the frontend, open the api webpage.
**Test Data Summary:** Meat ingredient is in inventory, Lettuce ingredient is in inventory, Bun ingredient is in inventory

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 2] 3.2.1.3 Test Add Invalid Item Price** | Preconditions: User is logged in as a staff member<br>1. Navigate to “Menu” page<br>2. Click “Add Item to Menu”<br>3. Type “InvalidBurger” for name, -500 for price, and 1 Meat, 1 Lettuce, 1 Bun<br>4. Submit the form, and check results | The form should not submit, as there was an invalid unit for price | The system delivers the prompt, “Price must be valid integer.” |

**Testing Overview:** Launch the backend, launch the frontend, open the api webpage.
**Test Data Summary:** Meat ingredient is in inventory, Lettuce ingredient is in inventory, Bun ingredient is in inventory. Burger from 3.2.1.1 exists.

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 3] 3.2.2.1 Test Delete Recipe Unused** | Preconditions: User is logged in as a staff member, 3.2.1.1 has passed. A recipe named “Burger” with arbitrary ingredients exists<br>1. Navigate to “Recipes” page<br>2. Click “Delete” next to Burger<br>3. Refresh page | The Burger menu item should no longer exist in the website | The Burger menu item should no longer exist in the website |

**Testing Overview:** Launch the backend, launch the frontend, open the api webpage.
**Test Data Summary:** Meat ingredient is in inventory, Lettuce ingredient is in inventory, Bun ingredient is in inventory. Burger from 3.2.1.1 exists. An unfulfilled order has been placed for the burger

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 3] 3.2.2.2 Test Delete Recipe already used** | Preconditions: User is logged in as a staff member, 3.2.1.1 has passed, 3.1.1 has passed<br>1. Navigate to “Recipes” page<br>2. Click “Delete” next to Burger<br>3. Check Results | An error should appear saying that there is an order for the burger in progress | The system delivers the prompt, “Recipe cannot be changed after existing in an order” |

**Testing Overview:** Launch the backend, launch the frontend, open the api webpage.
**Test Data Summary:** Meat ingredient is in inventory, Lettuce ingredient is in inventory, Bun ingredient is in inventory. Burger from 3.2.1.1 exists

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 3] 3.2.2.3 Test Delete Item In Progress** | Preconditions: User is logged in as a staff member, 3.2.1.1 has passed, 3.1.1 has passed. An item named “water bottle” exists and has been in an order<br>1. Navigate to “Items” page<br>2. Open a second tab, navigate to “Menu” page<br>3. Click “Delete” next to Burger<br>4. Click “Delete” on other tab<br>5. Check Results | An error should appear saying that the item has already been deleted | The system delivers the prompt, “Recipe cannot be changed after existing in an order” |

**Testing Overview:** Launch the backend, launch the frontend, open the api webpage.
**Test Data Summary:** Meat ingredient is in inventory, Lettuce ingredient is in inventory, Bun ingredient is in inventory. Burger from 3.2.1.1 exists

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 4 3.2.3.1 Test Edit Recipe Unused** | Preconditions: User is logged in as a staff member, 3.2.1.1 has passed, “Burger” recipes exists<br>1. Navigate to “Recipes” page<br>2. Click “Edit” next to Burger<br>3. Change price to 450, Meat to 2<br>4. Submit form, check results | The Burger item should now have a price of 450 and consume 2 Meat when created | The Burger item should now have a price of 450 and consume 2 Meat when created |

**Testing Overview:** Launch the backend, launch the frontend, open the api webpage.
**Test Data Summary:** Meat ingredient is in inventory, Lettuce ingredient is in inventory, Bun ingredient is in inventory. Burger from 3.2.1.1 exists

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 4] 3.2.3.2 Test Edit Invalid Price** | Preconditions: User is logged in as a staff member, 3.2.1.1 has passed<br>1. Navigate to “Menu” page<br>2. Click “Edit” next to Burger<br>3. Change price to -450<br>4. Submit form, check results | The form should not submit, as the price is an invalid amount | The system delivers the prompt, “Price must be a positive integer.” |

**Testing Overview:** Launch the backend, launch the frontend, open the api webpage.
**Test Data Summary:** Meat ingredient is in inventory, Lettuce ingredient is in inventory, Bun ingredient is in inventory. Burger from 3.2.1.1 exists

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 4] 3.2.3.3 Test Edit Invalid Ingredient** | Preconditions: User is logged in as a staff member, 3.2.1.1 has passed<br>1. Navigate to “Menu” page<br>2. Click “Edit” next to Burger<br>3. Change Meat to -1 | The form should not submit, as the ingredient Meat is an invalid amount | The system delivers the prompt, “Amount cannot be negative.” |

| | | | |
|---|---|---|---|
| | 4. Submit form, check results | | |

**Testing Overview:** The staff member login into the system: WolfCafe with valid credentials. The staff member is going to update the quantity of an existing ingredient in a valid manner.
**Test Data Summary:** The staff must have at least one ingredient in the system: coffee with a quantity of 15

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 5] 3.2.4.1 Test Valid Modify Inventory Ingredient Request** | Preconditions: User is logged in as a staff member, coffee exists in database<br>1. The User navigates to the Inventory tab<br>2. The user selects the ingredient coffee from the drop down list.<br>3. The user changes the units of coffee from 15 to 20<br>4. The user clicks submit | The inventory should now be modified with the amount of 20 for coffee | The inventory is modified with the amount of 20 for coffee |

**Testing Overview:** The staff member login into the system: WolfCafe with valid credentials The staff member is going to update the quantity of an existing ingredient in a invalid manner.
**Test Data Summary:** The staff must have at least one ingredient in the system: coffee with a quantity of 15

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 5] 3.2.4.2 Test Invalid Modify Inventory Ingredient Request** | Preconditions: User is logged in as a staff member, coffee exists in database<br>1. The User navigates to the Inventory tab<br>2. The user selects add Ingredient at the top<br>3. The user select the | An error message should display indicating a negative inventory amount is invalid | The system delivers the prompt, “coffee must be a positive integer. |

| | | | |
|---|---|---|---|
| | ingredient coffee<br>4. The user changes the units of coffee from 15 to -20<br>5. The user clicks submit | | |

**Testing Overview:** The staff member login into the system: WolfCafe with valid credentials. The staff member is going to update the quantity of multiple existing ingredients in a valid manner.
**Test Data Summary:** The staff should have multiple ingredients in the system: sugar, lemon, water, and ice all with an amount of 15

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 5] 3.2.4.3 Test Modify Multiple Inventory Ingredient Request** | Preconditions: The user is logged in as a staff member, and sugar, lemon, water, and ice exist in database<br>1. The User navigates to the Inventory tab<br>2. The user selects add Ingredient at the top<br>3. The user changes the units of sugar from 15 to 20<br>4. The user changes the units of lemon from 15 to 20<br>5. The user changes the units of water from 15 to 20<br>6. The user changes the units of ice from 15 to 20<br>7. The user clicks submit | The Inventory should not be updated with the proper amounts for the ingredients sugar, lemon, water, and ice. All the amounts should be 20 | The Inventory reflects the expected changes upon refresh. |

**Testing Overview:** The staff member should login into WolfCafe using proper credentials. The staff member is going to complete an unfulfilled order.
**Test Data Summary:** Test data from 3.1.1.1 exists as a valid pending order in the system

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 6] 3.2.5.1 Test Valid Complete Pending Order** | Preconditions: The user is logged in as a staff member, and test 3.1.1.1 is passing up till step 6 (is a valid pending order)<br>1. The staff user navigates to the pending orders tab<br>2. The staff member sees the 3.1.1.1 order in the orders tab<br>3. The staff member fulfills the order and changes its status from pending to ready. | The pending orders list is updated to show the order as ready for pickup. The user from test 3.1.1.1 can log in and view their order's status, which is changed from pending to ready in the web portal. | The order is listed as READY_FOR_PICKUP in the user view as expected and they have the option to click picked up order. |

**Testing Overview:** The staff member should login into WolfCafe using proper credentials
**Test Data Summary:** There must be unfulfilled orders already in the system. For instance there is an order for a burger and it is the end of the day so WolfCafe is now closed

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 6] 3.2.5.2 Valid End of Day Pending Order** | Preconditions: The user is logged in as a staff member, and test 3.2.5.1 is passing up<br>1. The staff user navigates to the pending orders tab at the end of the day<br>2. The staff member sees the 3.1.1.1 order in the orders tab which is marked as ready but not picked up<br>3. The staff member changes the status of the order to No Pickup and throws the order away | The pending orders list is updated to remove the order. The user from test 3.1.1.1 can log in and see that their order is deleted due to not being picked up. | The order is deleted from the user's view as expected. |

**Testing Overview:** The staff member should login into WolfCafe using proper credentials. The staff member is going to complete multiple unfulfilled orders after multiple customers place orders.
**Test Data Summary:** There must be unfulfilled orders already in the system. Complete test 3.1.1.1 using two or more different users

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 6] 3.2.5.3 Valid Complete Multiple Pending Orders** | Preconditions: The user is logged in as a staff member, and test 3.1.1.1 is passing using multiple different users.<br>1. The staff member see in the orders tab there are multiple pending orders for burgers from multiple users.<br>2. The staff member marksall of the orders as ready for pickups<br>3. Multiple users log in and check results | The selected orders are changed from pending orders to orders ready for pickup in the system. | All of the users who log in (including anonymous) can see their orders as ready for pickup. |

**Testing Overview:** A user logs in to the webpage as an admin and attempts to add a valid staff user to the system.
**Test Data Summary:** Information for new staff user: **Name**: Lebron James **Email**: lbjames@ncsu.edu **Username**: lbjames **Password**: t3stp@ssw0rd **Location:** Location created

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 7] 3.3.1.1 Test Add New Staff User to System** | Preconditions: Frontend and backend are running. A valid location has been added to the system. The user opens the webpage and logs in as an admin user.<br>1. Admin user navigates to the create staff page option in the header<br>2. Admin enters the | The user is returned to the view users page and the new staff member is displayed. | User returns to the view users page and sees the list of users with the bottom one being the “Lebron James” the admin just created with the provided test data. |

| | | | |
|---|---|---|---|
| | provided test data into the relevant fields in the add staff user page<br>3. Admin clicks the “Submit” button at the bottom of the page. | | |

**Testing Overview:** A user logs in to the webpage as an admin and attempts to add a staff user with incomplete information to the system.
**Test Data Summary:** Information for new staff user: **Name**: [blank] **Email**: mjjordan@ncsu.edu **Username**: mjjordan **Password**: t3stp@ssw0rd **Location:** Location created

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 7] 3.3.1.2 Test Add New Staff User to System (Incomplete information)** | Preconditions: Frontend and backend are running. The user opens the webpage and logs in as an admin user.<br>1. Admin user navigates to the create staff page option in the header<br>2. Admin enters the provided test data into the relevant fields in the add staff user page<br>3. Admin clicks the “Submit” button at the bottom of the page and checks results. | The system shows the user a popup with the message “All fields are required, including location.<br>” and will continue until all fields have valid input. | The system does the user a popup with the message “All fields are required, including location.<br>” and not add a user unless all fields have valid input. |

**Testing Overview:** A user logs in to the webpage as an admin and attempts to add a duplicate staff user to the system.
**Test Data Summary:** Information for new staff user: **Name**: Lebron Jahamez **Email**: lbjames@ncsu.edu **Username**: ljames **Password**: t3stp@ssw0rd

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **[UC 7] 3.3.1.3 Test Add New Staff User to System (Duplicate information)** | Preconditions: Test 3.3.1.1 has already passed<br>1. Admin user navigates to the create staff page option in the header<br>2. Admin enters the provided test data into the relevant fields in the add staff user page<br>3. Admin clicks the “Submit” button at the bottom of the page. | The system shows the user a popup with the message “Failed to create staff user.” and the user will not be created until non-duplicate details are provided. | The system displays the expected “Failed to create staff user” and upon return to the view users page the duplicate member is not created. |

**Testing Overview:** A user logs in to the webpage as an admin and attempts to remove a valid staff user from the system.
**Test Data Summary:** The user created in Test 3.3.1.1 has been created and exists

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 8] 3.3.2.1 Test Remove Staff User from System** | Preconditions: Test 3.3.1.1 has already passed<br>1. Admin user navigates to the view users page<br>2. Next to the entry of the staff created in 3.3.1.1 admin selects the delete staff user.<br>3. Admin clicks confirm in the confirmation dialog and confirms the results | The system refreshes with an updated user list and displays the user with the deleted user removed. | The system refreshes with an updated user list and displays the list with the expected user removed In the db, all references to the user in users and users_roles are removed. |

**Testing Overview:** A user logs in to the webpage as an admin and attempts to remove a staff user already removed from the system.
**Test Data Summary:** The user created in Test 3.3.1.1 has been created and exists

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **[UC 8] 3.3.2.2 Test Remove Staff User from System (User already removed)** | Preconditions:<br>1. Admin user navigates to the view users page and views user created in 3.3.1.1<br>2. Admin opens another edit staff window in a new tab and runs test 3.3.2.1<br>3. Admin returns to the original window and clicks remove user next to the user entry.<br>4. Admin clicks confirm in the confirmation dialog and confirms the results | The system shows the user a message “Failed to delete user” and upon refresh displays a list without the deleted user. | The system shows the user a message “Failed to delete user” and upon refresh displays a list without the deleted user. The remaining users remain untouched. |

**Testing Overview:** A user logs in to the webpage as an admin and attempts to remove the admin user.
**Test Data Summary:** Test 3.1.1.1 has already passed in another window (but stopped on step 6 to keep order pending)

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 8] 3.3.2.3 Test Remove Admin User from System** | Preconditions:<br>1. Admin user navigates to the view user page and clicks the delete admin user and checks results | The system shows the user a message “Cannot delete admin user” and the admin user will not delete even upon refresh. | The system shows the user a message “Cannot delete admin user” and the admin user does not delete even upon refresh. |

**Testing Overview:** An administrator is able to modify a user’s balance in the system
**Test Data Summary:** An account entry with a name “John”, role “customer” and valid user info.

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **[UC 9] 3.3.3.1 Test: Modify Customer user in system (Valid Test)** | Preconditions:<br>1. The user is logged in as an admin in the WolfCafe system<br>2. An account exists named “John”, role “customer”.<br><br>Main Flow:<br>1. The admin navigates from the dashboard to the user list<br>2. The admin selects the Modify button next to the Customer entry “John” entry<br>3. The admin is taken to an edit user page where they are able to edit the entry details<br>4. The admin clicks the “Save Details” button at the bottom of the page to save the details of the entry. The admin is returned to the list they were in with the updated entry displayed. | The user correlating with John’s position in the list has been changed to reflect the new information. | The user correlating with John’s position in the list will have changed information based on what is entered in the modify page. |

**Testing Overview:** An administrator should not be able to cause duplicate email entries within two unique user accounts through user account modification in the administrative dashboard.
**Test Data Summary:** An account entry of ID 1 with a name “John”, ID 1, email “jkaczma@ncsu.edu”, role “customer”. Another account entry of ID 2 with a anime “Daniel”, ID 2, email “danielk@ncsu.edu”, role “staff”

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **[UC 9] 3.3.3.2 Test: Email conflict due to duplicate entry in user profile update (Invalid Test)** | Preconditions:<br>1. The user is logged in as an admin in the WolfCafe system<br>2. An account exists with the name “John”, email “jkaczma@ncsu.edu” role “user”,<br>3. Another account exists with the name “Daniel”, email “danielk@ncsu.edu” role “user”<br><br>Main Flow:<br>1. The admin navigates from the dashboard to the view users list<br>2. The admin selects the Modify button next to the “John” entry<br>3. The admin is taken to an edit user page where they can modify the entry details<br>4. The admin selects the email field and changes the email from “jkaczma@ncsu.edu” to “danielk@ncsu.edu”<br>5. The admin clicks the “Save Details” button at the bottom of the page to save the changes. | An error message is displayed, stating “Failed to update user.” | An error message is displayed, stating “Failed to update user.” Upon return to view users page, the email is left unchanged for user John. |

**Testing Overview:** An administrator should be able to add a location with tax within the legal constraints of a 2-50% interval.
**Test Data Summary:** Name: Raleigh Address: “kdkdjsfsdkjf Tax Rate: 6%

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 9] 3.3.4.1: Edit System Tax Rate (Valid Test)** | Preconditions:<br>1. The user is logged in as an admin in the WolfCafe system. Test 3.3.4.1 is passing<br>Main Flow:<br>1. The admin navigates from the dashboard to the “Locations” option.<br>2. The admin enters the name “Raleigh” address as a string, and tax rate at .06, and End of Day time as 6:00 PM.<br>3. The admin clicks the “Save” button to create a location with the given tax rate.<br>4. The admin edits the .06 in the locations table to .02 and presses submit.<br>5. The page references, now displaying the updated tax rate from within the system | The system shows the message, “Tax rate updated successfully” and the updated location with the provided tax rate. Orders to this location will use this tax rate when applying tax to orders. | Raleigh’s tax rate is changed to .02 even upon refresh of the page. |

**Testing Overview:** An administrator should be able to add a location with tax within the legal constraints of a 2-50% interval.

**Test Data Summary:** Name: Raleigh Address: “kdkdjsfsdkjf Tax Rate: -6%

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 9] 3.3.4.2: Edit System Tax Rate (Negative amount)** | Preconditions:<br>2. The user is logged in as an admin in the WolfCafe system. Test 3.3.4.1 is | A message displays, “Tax rate must be at least 2%” | A message displays, “Tax rate must be at least 2%” |

| | | | |
|---|---|---|---|
| | passing<br>Main Flow:<br>1. The admin navigates from the dashboard to the “Locations” option.<br>2. The admin edits the tax rate at Raleigh to be at .01<br>3. The admin clicks the “Submit” button. | | |

**Testing Overview:** An administrator should be able to update the system’s tax rate within the legal constraints of a 2% minimum.
**Test Data Summary:** Name: Raleigh Address: “kdkdjsfsdkjf Tax Rate: 1%

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 9] 3.3.4.3: Edit System Tax Rate (<2%)** | Preconditions:<br>The user is logged in as an admin in the WolfCafe system. Test 3.3.4.1 is passing<br>Main Flow:<br>4. The admin navigates from the dashboard to the “Locations” option.<br>5. The admin edits the tax rate at Raleigh to be at .01<br>6. The admin clicks the “Submit” button. | A message displays, “Tax rate must be at least 2%” | A message displays, “Tax rate must be at least 2%” |

**Testing Overview:** The staff member should be able to access the Developer’s guide, ensuring it is accessible.
**Test Data Summary:** The accessing user account should meet the following requirements: role: staff

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **[UC 10] 3.4.1.1 Test: Staff views the developer's guide (Valid Test)** | Preconditions:<br>1. The user is logged in as a staff member in the Wolf Cafe system.<br>2. The user is on the main page.<br><br>Main Flows:<br>1. The user scrolls down to the bottom of the page.<br>2. The user clicks on the hyperlink "View Developer's Guide"<br>3. The system shows the Developer's Guide resource. | The Developer's guide is shown successfully. | The Developer's guide is shown successfully. |

**Testing Overview:** User opens the app and attempts to search for the developer's guide (perhaps in an attempt to exploit the system)
**Test Data Summary:** None, as the user is not logged in, and is attempting to access a secure resource.

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 10] 3.4.1.2 Test: Customer views the developer's guide (Invalid test)** | Preconditions:<br>1. The user is on the main page in the user portion of the Wolf Cafe system.<br><br>Main Flow:<br>1. The customer uses the search bar to type "Developer's Guide"<br>2. The customer presses the search button.<br>3. The system processes | The developer's guide button redirects to the menu for customer users. | The developer's guide redirects to the menu for customer users as expected. |

| | | | |
|---|---|---|---|
| | the search query and displays the results to the customer. | | |

**Testing Overview:** A user, not associated with a user account, attempts to view the privacy policy.
**Test Data Summary:** None, as the user is not logged in.

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 10] 3.4.1.3 Test: Customer views the privacy policy (valid test)** | Preconditions:<br>1. The user is logged in as a staff member in the Wolf Cafe system.<br>2. The user is on the main page.<br><br>Main Flows:<br>1. The user scrolls down to the bottom of the page.<br>2. The user clicks on the hyperlink “View Privacy Policy”<br>3. The system shows the privacy policy resource. | The privacy policy document is loaded in the same page to the user, without any errors / access restrictions.<br><br>The document is clear and accessible to the user. | The privacy policy document is loaded in the same page to the user, without any errors / access restrictions.<br><br>The document is clear and accessible to the user. |

**Testing Overview:** User opens the app.
**Test Data Summary:** No prerequisite**.**

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 11] 3.5.1.1 Test Valid Account** | Preconditions: Valid location data has been added to the system by an admin user<br>1. Navigate to “Sign Up” page<br>2. Insert “Andrew” for name “aaparr” for username<br>3. Insert “aaparr@ncsu.edu” for “email”<br>4. Insert “hunter2” for “password” and “repeat password”<br>5. For location input name of valid location that admin created<br>6. Select “Sign Up” and check results. | Forwarded to “Login” page with a new customer user filled with name: Andrew, username: aaparr email: aaparr@ncsu.edu, and password: hunter2. Should be able to login with valid user. | The valid Andrew user is created and the login credentials are validated. When logged in as Andrew user can be seen with details. |

**Testing Overview:** User opens the app.
**Test Data Summary:** No prerequisite**.**

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 11] 3.5.1.2 Test Password Mismatch** | Preconditions: Valid location data has been added to the system by an admin user<br>1. Navigate to “Sign Up” page<br>2. Insert “Andrew” for “username”<br>3. Insert “aaparr@ncsu.edu” for “email”<br>4. Insert “hunter2” for “password”<br>5. Insert “hunter3” for “repeat password”<br>6. For location input name of valid location that admin created<br>7. Select “Sign Up” and check results. | Message saying “Passwords do not match.” is displayed. | Message saying “Passwords do not match.” is displayed. User is not added to system. |

**Testing Overview:** User opens the app.

**Test Data Summary:** User of username: Andrew, email: aaparr@ncsu.edu, and password: hunter2 exists already**.**

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 11] 3.5.1.3 Test Duplicate Email** | Preconditions: User Andrew is already created.<br>1. Navigate to “Sign Up” page<br>2. Insert “Andrew” for “username”<br>3. Insert “aaparr@ncsu.edu” for “email”<br>4. Insert “hunter2” for “password” and “repeat password”<br>5. Select “Sign Up” and check results. | Message saying “Failed to register. Please try again.” is displayed. | Message saying “Failed to register. Please try again.” is displayed. User is not added to system. |

**Testing Overview:** Backend and frontend are both running. A user will attempt to log in to their account
**Test Data Summary:** A user exists exists in the system with email: **aaparr@ncsu.edu** and password: **hunter2**

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 12] 3.5.2.1 Test Valid Log On/Off** | Preconditions: 3.5.1.1 has passed<br>1. The user enters aaparr@ncsu.edu in email and **hunter2** in password<br>2. The user selects “log in”<br>3. The user is taken to the home page of the website<br>4. The user selects “log out”<br>5. A popup asks if they are sure, the user selects “yes” | The user should see the home page before they log out, and after they log out, they should be returned to the login page | The user should see the home page before they log out, and after they log out, they should be returned to the login page |

**Testing Overview:** Backend and frontend are both running. A user will attempt to log in to their account
**Test Data Summary:** A user exists exists in the system with email: **aaparr@ncsu.edu** and password: **hunter2**

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 12] 3.5.2.2 Test Invalid** | Preconditions: 3.5.1.1 has passed | The login page should not | The login page does not repond |

| | | | |
|---|---|---|---|
| **Email** | 1. The user enters rngallo@ncsu.edu in email and **extruck14** in password<br>2. The user selects log in<br>3. Check results | respond to the invalid input. | to the invalid input. |

**Testing Overview:** Backend and frontend are both running. A user will attempt to log in to their account

**Test Data Summary:** A user exists exists in the system with email: **aaparr@ncsu.edu** and password: **hunter2**

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 12] 3.5.2.3 Test Invalid Password** | Preconditions: 3.5.1.1 has passed<br>4. The user enters aaparr@ncsu.edu in email and **extruck14** in password<br>5. The user selects log in<br>6. Check results | The login page should not respond to the invalid input. | The login page does not repond to the invalid input. |

**Testing Overview:** The customer navigates to confirm pickup of latest order. Staff members are working.

**Test Data Summary:** Customer is logged in. Ordered a Burger item is on the menu at $10, insert “50” in “Balance”, and selected 20% tip.

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 13] 3.1.2.1 Test Valid Customer Pickup** | Preconditions: User is logged in as a customer and placed order<br>1. Navigate to “Order Log” page<br>2. List of active orders is displayed with “Picked Up Order” buttons<br>3. “Burger” order is “Status: Pending”<br>4. Staff member fulfills it.<br>5. “Burger” order is “Status: READY_FOR_PICKUP” | Order “Burger” is shown as "Status: COMPLETED" and enters the "Order History" tab. | The order with the “Burger” is shown as "Status: COMPLETED" and enters the "Order History" tab. |

| | | | |
|---|---|---|---|
| | 6. Select “Pickup Order” next to “Burger” and check results | | |

**Testing Overview:** The customer navigates to confirm pickup of latest order. Staff members are working.
**Test Data Summary:** Customer is logged in. Ordered a Burger item is on the menu at $10, insert “50” in “Balance”, and selected 20% tip.

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 13] 3.1.2.2 Test Order-Cancelled Customer Pickup** | Preconditions: User is logged in as a customer and placed order<br>1. Navigate to “Order Log” page<br>2. List of active orders is displayed with “Picked Up Order” buttons<br>3. “Burger” order is “Status: Pending”<br>4. Staff member deletes the order and customer checks results | Order “Burger” is deleted from the system and staff and customer can no longer see it. | Order “Burger” is deleted from the system and staff and customer can no longer see it. |

**Testing Overview:** The customer never navigates to confirm pickup of latest order. Staff members are working.
**Test Data Summary:** Customer is logged in. Ordered a Burger item is on the menu at $10, insert “50” in “Balance”, and selected 20% tip.

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 13] 3.1.2.3 Test No-Pickup Customer Pickup** | Preconditions: User is logged in as a customer and placed order<br>1. Navigate to “Order Log” page<br>2. “Burger” order is “Status: Pending”<br>3. **Reach the End of Day** wait approximately 10 minutes refresh the page and check results | Once the system thread handling end of day refreshes the order is removed from the system and staff and customer can no longer see it. | Once the thread handles eod the page refreshes and the order is removed from staff and customer view. |

**Testing Overview:** Staff navigates to add new ingredient to the inventory.
**Test Data Summary:** Staff is logged in and plans to add ingredient “Meat.”

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 14] 3.2.6.1 Test Valid Add New Ingredients** | Preconditions: User is logged in as staff<br>1. Navigate to “Add Ingredients” page<br>2. Enter name: “Meat”<br>3. Enter amount: “50”<br>4. Select “Add Ingredient” and check results | The user is redirected back to the ingredients list and the new ingredient is shown with the correct quantity. | The user is redirected to the ingredients list and can see the meat ingredient with a quantity of 50. |

**Testing Overview:** Staff navigates to add new ingredient to the inventory.
**Test Data Summary:** Staff is logged in and plans to add ingredient “Meat.”

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 14] 3.2.6.2 Test Invalid-Unit Add New Ingredients** | Preconditions: User is logged in as staff<br>1. Navigate to “Add Ingredients” page<br>2. Enter name: “Meat”<br>3. Enter amount: “-50”<br>4. Select “Submit” and check results | A message displays saying, “Invalid amount to add to the inventory (must be positive integer)” | A message displays saying, “Invalid amount to add to the inventory (must be positive integer)” |

**Testing Overview:** Staff navigates to add new ingredient to the inventory.
**Test Data Summary:** Staff is logged in and plans to add ingredient “Meat.”

| Test ID | Test Description | Expected Results | Actual Results |
|---|---|---|---|
| **[UC 14] 3.2.6.3 Test Duplicate Add New Ingredients** | Preconditions: User is logged in as staff and has already added ingredient: Meat to the inventory<br>1. Navigate to “Add Ingredients” page<br>2. Enter name: “Meat”<br>3. Enter amount: “50”<br>4. Select “Add Ingredient” and | A message displays saying, “Ingredient name already exists.” | The message. “Ingredient name already exists.” displays as expected. |

| | | | |
|---|---|---|---|
| | check results | | |